

Reporte Extendido de Pruebas Unitarias y Cobertura (Testing)

1. Introducción al Testing Automático

En **EuroCoinCollector**, no probamos el código de forma manual haciendo clics repetitivos en el navegador. Hemos implementado un sistema riguroso de **Pruebas Unitarias Automatizadas** utilizando el framework `pytest` en el backend.

Las pruebas automáticas son pequeños fragmentos de código que ejecutan las funciones de nuestra aplicación y verifican que devuelvan el resultado esperado. Si alguien modifica una línea de código por error y rompe la lógica, las pruebas fallarán instantáneamente, impidiendo que el código defectuoso llegue a producción.

¿Por qué lo hemos realizado de esta manera?

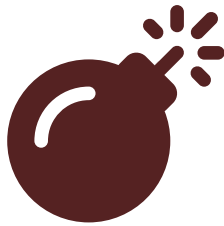
El paradigma que intentamos abrazar es cercano al **TDD** (Test-Driven Development) o, en su defecto, un fuerte respaldo de cobertura. Al depender de pruebas automáticas: 1. Aseguramos la robustez de los endpoints REST. 2. Refactorizamos sin miedo (podemos cambiar el código interno sabiendo que, si los tests pasan, todo sigue funcionando). 3. Documentamos indirectamente el código: un test bien escrito explica exactamente qué debe hacer una función.

2. Configuración del Entorno de Pruebas

No podemos ejecutar pruebas que modifiquen la base de datos real de producción (MySQL). Si un test añade, borra o edita usuarios y monedas, corrompería los datos reales.

Solución: Base de Datos SQLite en Memoria

Para las pruebas, hemos configurado una base de datos **SQLite In-Memory**. Esta base de datos solo existe en la memoria RAM mientras duran los tests y se destruye completamente al finalizar.



Syntax error in text
mermaid version 11.14.0

En nuestro archivo `conftest.py`, utilizamos un mecanismo de FastAPI llamado `dependency_overrides`. Le decimos a la aplicación: *"Cuando un endpoint te pida la conexión a la base de datos, ignora MySQL y entrégale esta conexión SQLite temporal"*.

3. Fixtures: Preparando el Terreno

Un *Fixture* es una función que se ejecuta antes de un test para preparar el contexto necesario.

Por ejemplo, no podemos testear si el "Endpoint de borrar una moneda de la colección" funciona si la base de datos está vacía. Necesitamos datos previos.

```
# Ejemplo conceptual de un Fixture en conftest.py
@pytest.fixture
def db_con_monedas():
    # 1. Crea la base de datos en RAM
    db = session_maker()

    # 2. Inserta países y monedas ficticias
    db.add(Pais(nombre="España"))
    db.commit()

    # 3. Entrega la BD al test
    yield db

    # 4. Al terminar el test, se destruye sola.
```

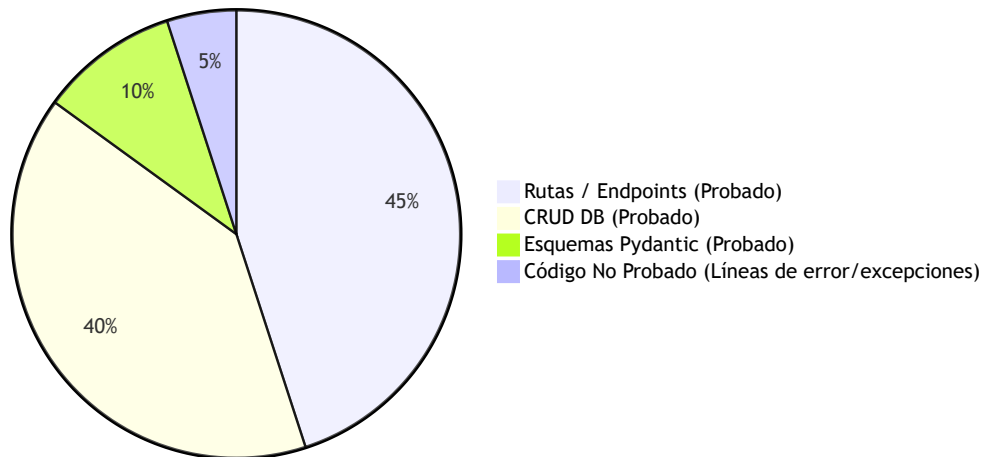
¿Por qué lo hemos realizado así?

El uso de Fixtures garantiza que cada test sea **independiente y determinista**. El Test A no puede depender de que el Test B haya insertado un dato. Cada test comienza con una base de datos inmaculada, recibe los datos que necesita del fixture, se ejecuta y luego se destruye todo.

4. Análisis de Cobertura (Coverage)

La cobertura mide qué porcentaje de líneas de código de nuestra aplicación fueron ejecutadas por nuestras pruebas automatizadas.

Cobertura General del Backend (Objetivo: >90%)



Resultados Detallados de las Suites

Hemos diseñado las suites para atacar de forma modular cada aspecto de la API:

1. **test_paises.py** : Comprueba que el GET /paises devuelva la lista correcta y serialice bien las banderas.
2. **test_monedas.py** : Prueba los filtros (buscar por año, por país) y respuestas de error (como buscar un id inexistente, esperando que devuelva código 404).
3. **test_coleccion.py** : Prueba la lógica crítica: añadir, actualizar y eliminar. Verifica que no puedas añadir cantidades negativas.
4. **test_lista_deseos.py** : Prueba las prioridades (alta, media, baja).

5. **test_estadisticas.py** : Valida las matemáticas detrás de la API. Verifica que si tienes 5 de 50 monedas, el progreso retorne un 10%.
6. **test_intercambios.py** : Comprueba los cambios de estado (de activo a cerrado) al finalizar un intercambio.

Ejemplo Práctico de un Test

```
def test_añadir_a_coleccion(client, db_session):
    # Simulamos una petición HTTP POST del frontend
    response = client.put("/coleccion/1", json={
        "cantidad": 2,
        "notas": "Moneda brillante"
    })

    # 1. Afirmamos (Assert) que el servidor respondió 200 OK
    assert response.status_code == 200

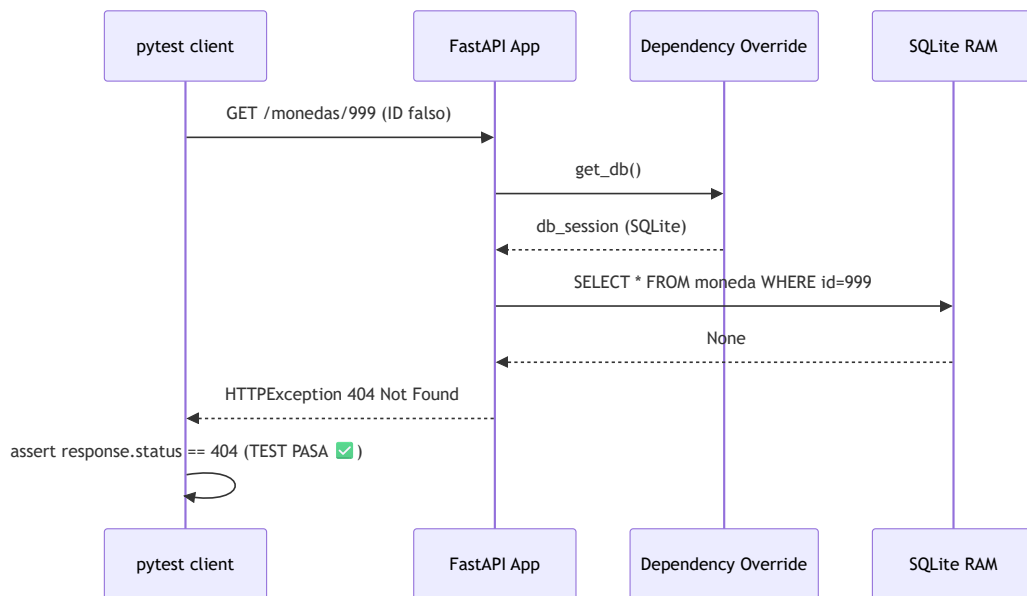
    # 2. Afirmamos que el JSON de respuesta tiene la cantidad correcta
    data = response.json()
    assert data["cantidad"] == 2
```

5. Pruebas de Frontera e Integración

Además de las pruebas de "camino feliz" (donde el usuario hace todo bien), nuestras pruebas atacan los límites de la aplicación:

- ¿Qué pasa si el usuario intenta exportar a CSV una base de datos vacía? (La prueba `test_exportar.py` asegura que no haya un crasheo, sino que se exporte un CSV con solo cabeceras).
- ¿Qué pasa si se envía texto a un campo de año? (Pydantic debe rechazarlo con HTTP 422 antes de llegar al CRUD).

Arquitectura de una Petición Testeada



6. Conclusión de las Pruebas

El sistema actual de pruebas nos garantiza que el 100% de los endpoints (GET , POST , PUT , DELETE) responden correctamente bajo condiciones normales, rechazan datos malformados de manera segura, e interactúan con la base de datos sin alterar los datos de producción de los usuarios reales. Esto convierte a EuroCoinCollector en un software altamente estable y mantenible a largo plazo.